Table Structure Example:

Let’s assume we have the following two tables:

1. employees table:

```
CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    hire_date DATE,
    department_id INT,
    salary DECIMAL(10, 2)
);
```

2. departments table:

```
CREATE TABLE departments (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(100)
);
```

INSERT INTO departments (department_id, department_name)

VALUES

(1, 'Sales'),

(2, 'Marketing'),

(3, 'Engineering'),

(4, 'HR'),

(5, 'Finance');

INSERT INTO employees (employee_id, first_name, last_name, hire_date, department_id, salary) VALUES

(1, 'John', 'Doe', '2015-06-23', 1, 55000.00),

(2, 'Jane', 'Smith', '2018-02-10', 2, 62000.00),

(3, 'Samuel', 'Adams', '2012-11-04', 3, 90000.00),

(4, 'Emily', 'Clark', '2020-03-15', 1, 45000.00),

(5, 'Daniel', 'Harris', '2016-07-19', 4, 49000.00),

(6, 'Rachel', 'Baker', '2019-10-01', 3, 95000.00),

(7, 'Paul', 'Jones', '2017-09-13', 2, 55000.00),

(8, 'Sophia', 'Taylor', '2014-12-21', 5, 73000.00),

(9, 'Michael', 'Lee', '2011-08-14', 4, 47000.00),

(10, 'Olivia', 'King', '2022-01-30', 3, 98000.00);

1) To concat the first_name and last_name of employees into a single column named full_name.

```
mysql> SELECT FIRST_NAME, LAST_NAME, SALARY
    -> FROM employees
    -> WHERE SALARY > (SELECT salary FROM employees WHERE last_name = 'Bull'
);
Empty set (0.01 sec)
```

2) Query that retrieves the first and last names of employees, as well as their department names by using a subquery inside the SELECT statement.

```
mysql> SELECT CONCAT(first_name, ' ', last_name) AS full_name
    -> FROM employees;
+---------------+
| full_name     |
+---------------+
| John Doe      |
| Jane Smith    |
| Samuel Adams  |
| Emily Clark   |
| Daniel Harris |
| Rachel Baker  |
| Paul Jones    |
| Sophia Taylor |
| Michael Lee   |
| Olivia King   |
+---------------+
10 rows in set (0.00 sec)
```

3) Query counts the number of employees in each department.

```
mysql> SELECT
    ->     first_name,
    ->     last_name,
    ->     (SELECT department_name
    ->      FROM departments
    ->      WHERE departments.department_id = employees.department_id) AS de
partment_name
    -> FROM employees;
+------------+-----------+-----------------+
| first_name | last_name | department_name |
+------------+-----------+-----------------+
| John       | Doe       | Sales           |
| Jane       | Smith     | Marketing       |
| Samuel     | Adams     | Engineering     |
| Emily      | Clark     | Sales           |
| Daniel     | Harris    | HR              |
| Rachel     | Baker     | Engineering     |
| Paul       | Jones     | Marketing       |
| Sophia     | Taylor    | Finance         |
| Michael    | Lee       | HR              |
| Olivia     | King      | Engineering     |
+------------+-----------+-----------------+
```

4) Query to lists all departments and counts the number of employees in each department .

```
mysql> SELECT
    ->     d.department_name,
    ->     COUNT(e.employee_id) AS employee_count
    -> FROM employees e
    -> JOIN departments d
    -> ON e.department_id = d.department_id
    -> GROUP BY d.department_name;
+-----------------+----------------+
| department_name | employee_count |
+-----------------+----------------+
| Engineering     |              3 |
| Finance         |              1 |
| HR              |              2 |
| Marketing       |              2 |
| Sales           |              2 |
+-----------------+----------------+
5 rows in set (0.00 sec)
```

5) To find employees with salary greater than average salary of their department

```
mysql> SELECT
    ->     e.first_name,
    ->     e.last_name,
    ->     e.salary,
    ->     e.department_id
    -> FROM employees e
    -> WHERE e.salary > (
    ->     SELECT AVG(e2.salary)
    ->     FROM employees e2
    ->     WHERE e2.department_id = e.department_id
    -> );
+------------+-----------+----------+---------------+
| first_name | last_name | salary   | department_id |
+------------+-----------+----------+---------------+
| John       | Doe       | 55000.00 |             1 |
| Jane       | Smith     | 62000.00 |             2 |
| Daniel     | Harris    | 49000.00 |             4 |
| Rachel     | Baker     | 95000.00 |             3 |
| Olivia     | King      | 98000.00 |             3 |
+------------+-----------+----------+---------------+
5 rows in set (0.01 sec)
```

6) To get all employee names in each department.

```
SELECT
    d.department_name,' at line 1
mysql> SELECT
    ->     d.department_name,
    ->     GROUP_CONCAT(CONCAT(e.first_name, ' ', e.last_name)) AS employee_
names
    -> FROM departments d
    -> LEFT JOIN employees e
    -> ON d.department_id = e.department_id
    -> GROUP BY d.department_name;
+----------------+-------------------------------------+
| department_name | employee_names                      |
+----------------+-------------------------------------+
| Engineering     | Samuel Adams,Olivia King,Rachel Baker |
| Finance         | Sophia Taylor                       |
| HR              | Michael Lee,Daniel Harris           |
| Marketing       | Paul Jones,Jane Smith               |
| Sales           | Emily Clark,John Doe                |
+----------------+-------------------------------------+
5 rows in set (0.01 sec)
```

7) Subquery to find the employee with the highest salary in each department.

```
mysql> SELECT
    ->     e.first_name,
    ->     e.last_name,
    ->     e.salary,
    ->     e.department_id
    -> FROM employees e
    -> WHERE e.salary = (
    ->     SELECT MAX(e2.salary)
    ->     FROM employees e2
    ->     WHERE e2.department_id = e.department_id
    -> );
+------------+-----------+----------+---------------+
| first_name | last_name | salary   | department_id |
+------------+-----------+----------+---------------+
| John       | Doe       | 55000.00 |             1 |
| Jane       | Smith     | 62000.00 |             2 |
| Sophia     | Taylor    | 73000.00 |             5 |
| Michael    | Lee       | 47000.00 |             4 |
| Olivia     | King      | 98000.00 |             3 |
+------------+-----------+----------+---------------+
5 rows in set (0.00 sec)
```

8) Group employees by hire year and calculate the total salary for each year

```
mysql> SELECT
    ->     YEAR(hire_date) AS hire_year,
    ->     SUM(salary) AS total_salary
    -> FROM employees
    -> GROUP BY YEAR(hire_date);
+-----------+--------------+
| hire_year | total_salary |
+-----------+--------------+
|      2011 |     47000.00 |
|      2012 |     90000.00 |
|      2014 |     73000.00 |
|      2015 |     55000.00 |
|      2017 |     55000.00 |
|      2018 |     62000.00 |
|      2019 |     95000.00 |
|      2020 |     45000.00 |
|      2022 |     98000.00 |
+-----------+--------------+
9 rows in set (0.01 sec)
```

9) Group departments and count the number of employees in each department

```
mysql> SELECT
    ->     d.department_name,
    ->     COUNT(e.employee_id) AS employee_count
    -> FROM departments d
    -> LEFT JOIN employees e
    -> ON d.department_id = e.department_id
    -> GROUP BY d.department_name;
+-----------------+----------------+
| department_name | employee_count |
+-----------------+----------------+
| Engineering     |              3 |
| Finance         |              1 |
| HR              |              2 |
| Marketing       |              2 |
| Sales           |              2 |
+-----------------+----------------+
5 rows in set (0.00 sec)
```

10) Find the highest salary in each department

```
mysql> SELECT
    ->     d.department_name,
    ->     MAX(e.salary) AS highest_salary
    -> FROM employees e
    -> JOIN departments d
    -> ON e.department_id = d.department_id
    -> GROUP BY d.department_name;
+-----------------+----------------+
| department_name | highest_salary |
+-----------------+----------------+
| Engineering     |       98000.00 |
| Finance         |       73000.00 |
| HR              |       47000.00 |
| Marketing       |       62000.00 |
| Sales           |       55000.00 |
+-----------------+----------------+
5 rows in set (0.00 sec)
```

11) Group employees by department and show the average salary in each department.

```
mysql> SELECT
    ->     d.department_name,
    ->     AVG(e.salary) AS avg_salary
    -> FROM employees e
    -> JOIN departments d
    -> ON e.department_id = d.department_id
    -> GROUP BY d.department_name;
+-----------------+--------------+
| department_name | avg_salary   |
+-----------------+--------------+
| Engineering     | 94333.333333 |
| Finance         | 73000.000000 |
| HR              | 47000.000000 |
| Marketing       | 58500.000000 |
| Sales           | 50000.000000 |
+-----------------+--------------+
5 rows in set (0.00 sec)
```

12) Find departments with more than 5 employees

```
mysql> SELECT
    ->     d.department_name,
    ->     COUNT(e.employee_id) AS employee_count
    -> FROM employees e
    -> JOIN departments d
    -> ON e.department_id = d.department_id
    -> GROUP BY d.department_name
    -> HAVING COUNT(e.employee_id) > 5;
Empty set (0.00 sec)
```

13) List departments where the average salary is greater than 50,000.

```
mysql> SELECT
    ->     d.department_name,
    ->     AVG(e.salary) AS avg_salary
    -> FROM employees e
    -> JOIN departments d
    -> ON e.department_id = d.department_id
    -> GROUP BY d.department_name
    -> HAVING AVG(e.salary) > 50000;
+-----------------+--------------+
| department_name | avg_salary   |
+-----------------+--------------+
| Engineering     | 94333.333333 |
| Finance         | 73000.000000 |
| Marketing       | 58500.000000 |
+-----------------+--------------+
3 rows in set (0.00 sec)
```

14) List employees who earn more than 60,000 and belong to a department with more than 3 employees.

```
mysql> SELECT
    ->     e.first_name,
    ->     e.last_name,
    ->     e.salary,
    ->     e.department_id
    -> FROM employees e
    -> WHERE e.salary > 60000
    -> AND e.department_id IN (
    ->     SELECT department_id
    ->     FROM employees
    ->     GROUP BY department_id
    ->     HAVING COUNT(*) > 3
    -> );
Empty set (0.00 sec)
```

15) Show departments where there is more than one employee with a salary over 100,000.

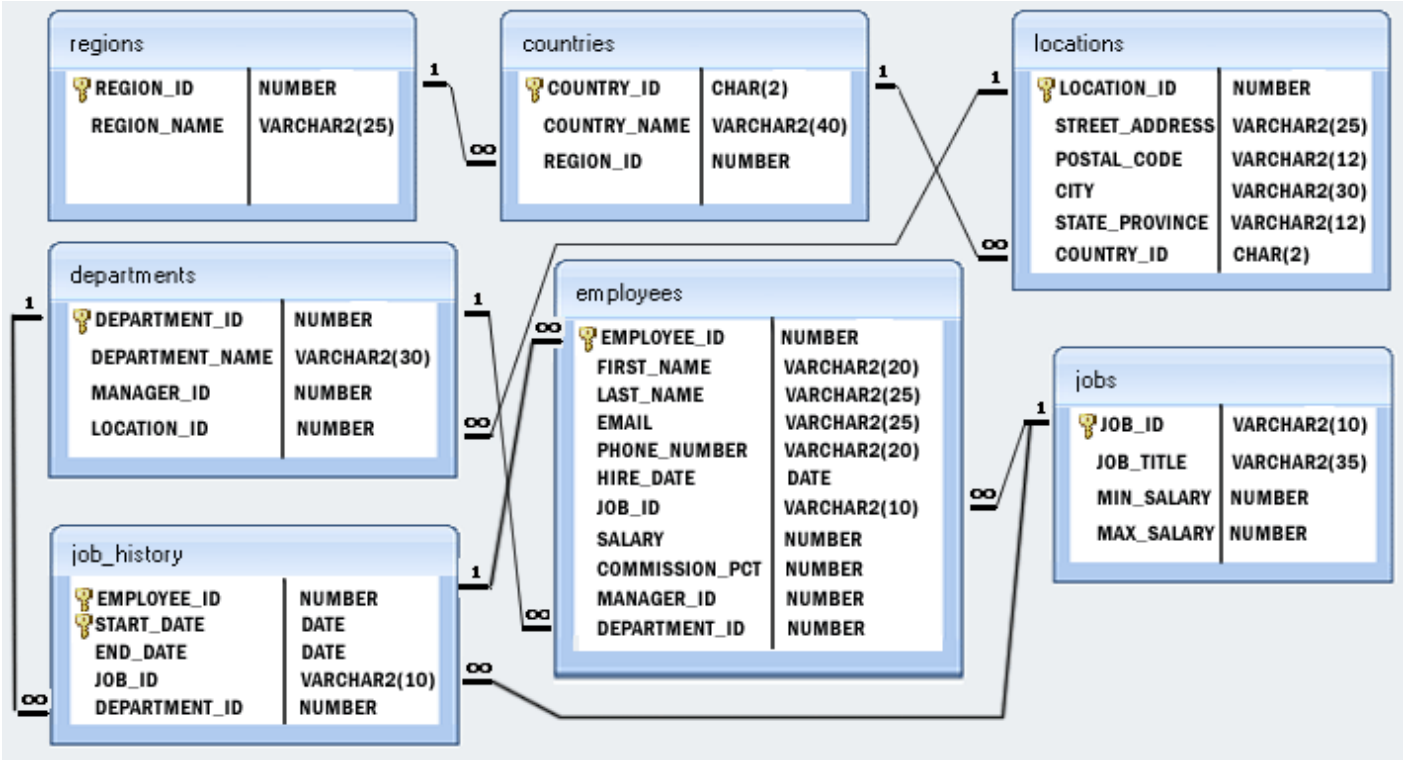
```
mysql> SELECT
    ->     d.department_name,
    ->     COUNT(e.employee_id) AS high_salary_count
    -> FROM employees e
    -> JOIN departments d
    -> ON e.department_id = d.department_id
    -> WHERE e.salary > 100000
    -> GROUP BY d.department_name
    -> HAVING COUNT(e.employee_id) > 1;
Empty set (0.00 sec)
```

16) Delete an employee with a specific employee id (e.g., 5)

```
mysql> DELETE FROM employees
    -> WHERE employee_id = 5;
Query OK, 1 row affected (0.01 sec)
```

17) Delete employees who have a salary lower than 30,000

```
mysql> DELETE FROM employees
    -> WHERE salary < 30000;
Query OK, 0 rows affected (0.00 sec)
```



MySQL Update Table exercises

1. Write a SQL statement to change the email column of employees table with 'not available' for all employees.

```
ysql> UPDATE employees
    -> SET email = 'not available';
uery OK, 9 rows affected (0.00 sec)
ows matched: 9  Changed: 9  Warnings: 0
```

2. Write a SQL statement to change the email and commission_pct column of employees table with 'not available' and 0.10 for all employees.

```
mysql> ALTER TABLE employees
    -> ADD commission_pct DECIMAL(5,2);
Query OK, 9 rows affected (0.03 sec)
Records: 9  Duplicates: 0  Warnings: 0

mysql> UPDATE employees
    -> SET
    ->     email = 'not available',
    ->     commission_pct = 0.10;
Query OK, 9 rows affected (0.01 sec)
Rows matched: 9  Changed: 9  Warnings: 0
```

3. Write a SQL statement to change the email and commission_pct column of employees table with 'not available' and 0.10 for those employees whose department_id is 110.

```
mysql> UPDATE employees
    -> SET
    ->     email = 'not available',
    ->     commission_pct = 0.10
    -> WHERE department_id = 110;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0
```

4. Write a SQL statement to change the email column of employees table with 'not available' for those employees whose department_id is 80 and gets a commission is less than .20%

```
mysql> UPDATE employees
    -> SET email = 'not available'
    -> WHERE department_id = 80
    -> AND commission_pct < 0.20;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0
```

5. Write a SQL statement to change the email column of employees table with 'not available' for those employees who belongs to the 'Accouning' department.

```
mysql> UPDATE employees
    -> SET email = 'not available'
    -> WHERE department_id IN (
    ->     SELECT department_id
    ->     FROM departments
    ->     WHERE department_name = 'Accounting'
    -> );
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0
```

6. Write a SQL statement to change salary of employee to 8000 whose ID is 105, if the existing salary is less than 5000.

```
mysql> UPDATE employees
    -> SET salary = 8000
    -> WHERE employee_id = 105
    -> AND salary < 5000;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0
```

7. Write a SQL statement to change job ID of employee which ID is 118, to SH_CLERK if the employee belongs to department, which ID is 30 and the existing job ID does not start with SH.

```
mysql> ALTER TABLE employees
    -> ADD job_id VARCHAR(20);
Query OK, 9 rows affected (0.02 sec)
Records: 9  Duplicates: 0  Warnings: 0

mysql> UPDATE employees
    -> SET job_id = 'SH_CLERK'
    -> WHERE employee_id = 118
    -> AND department_id = 30
    -> AND job_id NOT LIKE 'SH%';
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0
```

MySQL Alter Table exercises

1. Write a SQL statement to rename the table countries to country_new.

```
mysql> RENAME TABLE countries TO country_new;
Query OK, 0 rows affected (0.01 sec)
```

2. Write a SQL statement to add a column region_id to the table locations.

```
mysql> ALTER TABLE locations
    -> ADD region_id INT;
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

3. Write a SQL statement to add a columns ID as the first column of the table locations.

```
mysql> ALTER TABLE locations
    -> ADD id INT FIRST;
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

4. Write a SQL statement to add a column region_id after state_province to the table locations.

```
mysql> ALTER TABLE locations
    -> MODIFY region_id INT
    -> AFTER state_province;
Query OK, 0 rows affected (0.01 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

5. Write a SQL statement change the data type of the column country_id to integer in the table locations.

```
mysql> ALTER TABLE locations
    -> MODIFY country_id INT;
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

6. Write a SQL statement to drop the column city from the table locations.

```
mysql> ALTER TABLE locations
    -> DROP COLUMN city;
Query OK, 0 rows affected (0.01 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

7. Write a SQL statement to change the name of the column state_province to state, keeping the data type and size same.

```
mysql> ALTER TABLE locations
    -> CHANGE state_province state VARCHAR(25);
Query OK, 0 rows affected (0.01 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

8. Write a SQL statement to add a primary key for the columns location_id in the locations table.
Here is the sample table employees.
Sample table: employees

```
mysql> ALTER TABLE locations
    -> ADD PRIMARY KEY (location_id);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

9.Write a SQL statement to add a primary key for a combination of columns location_id and country_id.

```
mysql> ALTER TABLE locations
    -> ADD PRIMARY KEY (location_id, country_id);
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

10.Write a SQL statement to drop the existing primary from the table locations on a combination of columns location_id and country_id.

```
mysql> ALTER TABLE locations
    -> DROP PRIMARY KEY;
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Basic SELECT Statement Exercises

Sample table: employees

1. Write a query to display the names (first_name, last_name) using alias name "First Name", "Last Name"

```
mysql> SELECT
    ->     first_name AS "First Name",
    ->     last_name AS "Last Name"
    -> FROM employees;
+------------+-----------+
| First Name | Last Name |
+------------+-----------+
| John       | Doe       |
| Jane       | Smith     |
| Sam        | Adams     |
| Emily      | Clark     |
| Daniel     | Harris    |
| Rachel     | Baker     |
| Paul       | Jones     |
| Sophia     | Taylor    |
| Mike       | Brown     |
+------------+-----------+
9 rows in set (0.01 sec)
```

2. Write a query to get unique department ID from employee table.

```
mysql> SELECT DISTINCT department_id
    -> FROM employees;
+---------------+
| department_id |
+---------------+
|            60 |
|            80 |
|            40 |
|            50 |
|            30 |
+---------------+
```

3. Write a query to get all employee details from the employee table order by first name, descending.

```
mysql> SELECT *
    -> FROM employees
    -> ORDER BY first_name DESC;
+-------------+------------+-----------+---------+--------------+------------+----------+---------+----------------+------------+---------------+
| employee_id | first_name | last_name | email   | phone_number | hire_date  | job_id   | salary  | commission_pct | manager_id | department_id |
+-------------+------------+-----------+---------+--------------+------------+----------+---------+----------------+------------+---------------+
|         108 | Sophia     | Taylor    | STAYLOR | 123463       | 2014-12-21 | HR_REP   | 6500.00 |           NULL |       NULL |            40 |
|         103 | Sam        | Adams     | SADAMS  | 123458       | 2012-11-04 | IT_PROG  | 9000.00 |           NULL |       NULL |            60 |
|         106 | Rachel     | Baker     | RBAKER  | 123461       | 2019-10-01 | SA_REP   | 8000.00 |           0.20 |       NULL |            80 |
|         107 | Paul       | Jones     | PJONES  | 123462       | 2017-09-13 | SA_REP   | 5500.00 |           0.10 |       NULL |            80 |
|         118 | Mike       | Brown     | MBROWN  | 123470       | 2013-05-10 | IT_PROG  | 4000.00 |           NULL |       NULL |            30 |
|         101 | John       | Doe       | JDOE    | 123456       | 2015-06-23 | IT_PROG  | 6000.00 |           NULL |       NULL |            60 |
|         102 | Jane       | Smith     | JSMITH  | 123457       | 2018-02-10 | SA_REP   | 7000.00 |           0.15 |       NULL |            80 |
|         104 | Emily      | Clark     | ECLARK  | 123459       | 2020-03-15 | HR_REP   | 5000.00 |           NULL |       NULL |            40 |
|         105 | Daniel     | Harris    | DHARRIS | 123460       | 2016-07-19 | SH_CLERK | 3000.00 |           NULL |       NULL |            50 |
+-------------+------------+-----------+---------+--------------+------------+----------+---------+----------------+------------+---------------+
9 rows in set (0.01 sec)
```

4. Write a query to get the names (first_name, last_name), salary, PF of all the employees (PF is calculated as 12% of salary).

```
mysql> SELECT
    ->     first_name,
    ->     last_name,
    ->     salary,
    ->     salary * 0.12 AS PF
    -> FROM employees;
+------------+-----------+---------+-----------+
| first_name | last_name | salary  | PF        |
+------------+-----------+---------+-----------+
| John       | Doe       | 6000.00 |  720.0000 |
| Jane       | Smith     | 7000.00 |  840.0000 |
| Sam        | Adams     | 9000.00 | 1080.0000 |
| Emily      | Clark     | 5000.00 |  600.0000 |
| Daniel     | Harris    | 3000.00 |  360.0000 |
| Rachel     | Baker     | 8000.00 |  960.0000 |
| Paul       | Jones     | 5500.00 |  660.0000 |
| Sophia     | Taylor    | 6500.00 |  780.0000 |
| Mike       | Brown     | 4000.00 |  480.0000 |
+------------+-----------+---------+-----------+
9 rows in set (0.00 sec)
```

5. Write a query to get the employee ID, names (first_name, last_name), salary in ascending order of salary.

```
mysql> SELECT
    ->     employee_id,
    ->     first_name,
    ->     last_name,
    ->     salary
    -> FROM employees
    -> ORDER BY salary ASC;
+-------------+------------+-----------+---------+
| employee_id | first_name | last_name | salary  |
+-------------+------------+-----------+---------+
|         105 | Daniel     | Harris    | 3000.00 |
|         118 | Mike       | Brown     | 4000.00 |
|         104 | Emily      | Clark     | 5000.00 |
|         107 | Paul       | Jones     | 5500.00 |
|         101 | John       | Doe       | 6000.00 |
|         108 | Sophia     | Taylor    | 6500.00 |
|         102 | Jane       | Smith     | 7000.00 |
|         106 | Rachel     | Baker     | 8000.00 |
|         103 | Sam        | Adams     | 9000.00 |
+-------------+------------+-----------+---------+
```

6. Write a query to get the total salaries payable to employees.

```
mysql> SELECT
    ->     SUM(salary) AS total_salary
    -> FROM employees;
+--------------+
| total_salary |
+--------------+
|     54000.00 |
+--------------+
1 row in set (0.00 sec)
```

7. Write a query to get the maximum and minimum salary from employees table.

```
mysql> SELECT
    ->     MAX(salary) AS max_salary,
    ->     MIN(salary) AS min_salary
    -> FROM employees;
+------------+------------+
| max_salary | min_salary |
+------------+------------+
|    9000.00 |    3000.00 |
+------------+------------+
1 row in set (0.00 sec)
```

8. Write a query to get the average salary and number of employees in the employees table.

```
mysql> SELECT
    ->     AVG(salary) AS avg_salary,
    ->     COUNT(*) AS total_employees
    -> FROM employees;
+-------------+-----------------+
| avg_salary  | total_employees |
+-------------+-----------------+
| 6000.000000 |               9 |
+-------------+-----------------+
1 row in set (0.00 sec)
```

9. Write a query to get the number of employees working with the company.

```
mysql> SELECT
    ->     COUNT(*) AS total_employees
    -> FROM employees;
+-----------------+
| total_employees |
+-----------------+
|               9 |
+-----------------+
1 row in set (0.00 sec)
```

10. Write a query to get the number of jobs available in the employees table.

```
mysql> SELECT
    ->     COUNT(DISTINCT job_id) AS total_jobs
    -> FROM employees;
+------------+
| total_jobs |
+------------+
|          4 |
+------------+
1 row in set (0.01 sec)
```

MySQL Aggregate Functions and Group by- Exercises

<u>Sample table: employees</u>

1. Write a query to list the number of jobs available in the employees table.

```
mysql> SELECT COUNT(DISTINCT job_id) AS total_jobs
    -> FROM employees;
+------------+
| total_jobs |
+------------+
|          4 |
+------------+
1 row in set (0.00 sec)
```

2. Write a query to get the total salaries payable to employees.

```
mysql> SELECT SUM(salary) AS total_salary
    -> FROM employees;
+--------------+
| total_salary |
+--------------+
|     54000.00 |
+--------------+
1 row in set (0.00 sec)
```

3. Write a query to get the minimum salary from employees table.

```
mysql> SELECT MIN(salary) AS min_salary
    -> FROM employees;
+------------+
| min_salary |
+------------+
|    3000.00 |
+------------+
1 row in set (0.00 sec)
```

4. Write a query to get the maximum salary of an employee working as a Programmer.

```
mysql> SELECT MAX(salary) AS max_salary
    -> FROM employees
    -> WHERE job_id = 'IT_PROG';
+------------+
| max_salary |
+------------+
|    9000.00 |
+------------+
1 row in set (0.00 sec)
```

5. Write a query to get the average salary and number of employees working the department 90.

```
mysql> SELECT
    ->     AVG(salary) AS avg_salary,
    ->     COUNT(*) AS total_employees
    -> FROM employees
    -> WHERE department_id = 90;
+------------+-----------------+
| avg_salary | total_employees |
+------------+-----------------+
|       NULL |               0 |
+------------+-----------------+
1 row in set (0.00 sec)
```

6. Write a query to get the highest, lowest, sum, and average salary of all employees.

```
mysql> SELECT
    ->     MAX(salary) AS highest_salary,
    ->     MIN(salary) AS lowest_salary,
    ->     SUM(salary) AS total_salary,
    ->     AVG(salary) AS average_salary
    -> FROM employees;
+---------------+--------------+-------------+---------------+
| highest_salary | lowest_salary | total_salary | average_salary |
+---------------+--------------+-------------+---------------+
|        9000.00 |      3000.00 |    54000.00 |    6000.000000 |
+---------------+--------------+-------------+---------------+
1 row in set (0.00 sec)
```

7. Write a query to get the number of employees with the same job.

```
mysql> SELECT
    ->     job_id,
    ->     COUNT(*) AS employee_count
    -> FROM employees
    -> GROUP BY job_id;
+----------+----------------+
| job_id   | employee_count |
+----------+----------------+
| HR_REP   |              2 |
| IT_PROG  |              3 |
| SA_REP   |              3 |
| SH_CLERK |              1 |
+----------+----------------+
4 rows in set (0.00 sec)
```

MySQL Restricting and Sorting Data – Exercises

Sample table: employees

1. Write a query to display the names (first_name, last_name) and salary for all employees whose salary is not in the range 10,000 through 15,000.

```
mysql> SELECT
    ->     first_name,
    ->     last_name,
    ->     salary
    -> FROM employees
    -> WHERE salary NOT BETWEEN 10000 AND 15000;
+------------+-----------+---------+
| first_name | last_name | salary  |
+------------+-----------+---------+
| John       | Doe       | 6000.00 |
| Jane       | Smith     | 7000.00 |
| Sam        | Adams     | 9000.00 |
| Emily      | Clark     | 5000.00 |
| Daniel     | Harris    | 3000.00 |
| Rachel     | Baker     | 8000.00 |
| Paul       | Jones     | 5500.00 |
| Sophia     | Taylor    | 6500.00 |
| Mike       | Brown     | 4000.00 |
+------------+-----------+---------+
9 rows in set (0.00 sec)
```

2. Write a query to display the names (first_name, last_name) and department ID of all employees in departments 30 or 100 in ascending alphabetical order by department ID.

```
mysql> SELECT
    ->     first_name,
    ->     last_name,
    ->     department_id
    -> FROM employees
    -> WHERE department_id IN (30, 100)
    -> ORDER BY department_id ASC;
+------------+-----------+---------------+
| first_name | last_name | department_id |
+------------+-----------+---------------+
| Mike       | Brown     |            30 |
+------------+-----------+---------------+
1 row in set (0.01 sec)
```

3. Write a query to display the names (first_name, last_name) and salary for all employees whose salary is not in the range 10,000 through 15,000 and are in department 30 or 100.

```
mysql> SELECT
    ->     first_name,
    ->     last_name,
    ->     salary
    -> FROM employees
    -> WHERE salary NOT BETWEEN 10000 AND 15000
    -> AND department_id IN (30, 100);
+------------+-----------+---------+
| first_name | last_name | salary  |
+------------+-----------+---------+
| Mike       | Brown     | 4000.00 |
+------------+-----------+---------+
1 row in set (0.00 sec)
```

4. Write a query to display the names (first_name, last_name) and hire date for all employees who were hired in 1987.

```
mysql> SELECT
    ->     first_name,
    ->     last_name,
    ->     hire_date
    -> FROM employees
    -> WHERE YEAR(hire_date) = 1987;
Empty set (0.00 sec)
```

5. Write a query to display the first_name of all employees who have both "b" and "c" in their first name.

```
mysql> SELECT first_name
    -> FROM employees
    -> WHERE first_name LIKE '%b%'
    -> AND first_name LIKE '%c%';
Empty set (0.00 sec)
```

6. Write a query to display the last name, job, and salary for all employees whose job is that of a Programmer or a Shipping Clerk, and whose salary is not equal to 4,500, 10,000, or 15,000.

```
mysql> SELECT
    ->     last_name,
    ->     job_id,
    ->     salary
    -> FROM employees
    -> WHERE job_id IN ('IT_PROG', 'SH_CLERK')
    -> AND salary NOT IN (4500, 10000, 15000);
+-----------+----------+---------+
| last_name | job_id   | salary  |
+-----------+----------+---------+
| Doe       | IT_PROG  | 6000.00 |
| Adams     | IT_PROG  | 9000.00 |
| Harris    | SH_CLERK | 3000.00 |
| Brown     | IT_PROG  | 4000.00 |
+-----------+----------+---------+
4 rows in set (0.00 sec)
```

7. Write a query to display the last names of employees whose names have exactly 6 characters.

```
mysql> SELECT last_name
    -> FROM employees
    -> WHERE LENGTH(last_name) = 6;
+-----------+
| last_name |
+-----------+
| Harris    |
| Taylor    |
+-----------+
2 rows in set (0.01 sec)
```

8. Write a query to display the last names of employees having 'e' as the third character.

```
mysql> SELECT last_name
    -> FROM employees
    -> WHERE last_name LIKE '__e%';
+-----------+
| last_name |
+-----------+
| Doe       |
+-----------+
1 row in set (0.00 sec)
```

9. Write a query to display the jobs/designations available in the employees table.

```
mysql> SELECT DISTINCT job_id
    -> FROM employees;
+----------+
| job_id   |
+----------+
| IT_PROG  |
| SA_REP   |
| HR_REP   |
| SH_CLERK |
+----------+
4 rows in set (0.00 sec)
```

10. Write a query to display the names (first_name, last_name), salary and PF (15% of salary) of all employees.

```
mysql> SELECT
    ->     first_name,
    ->     last_name,
    ->     salary,
    ->     salary * 0.15 AS PF
    -> FROM employees;
+------------+-----------+---------+-----------+
| first_name | last_name | salary  | PF        |
+------------+-----------+---------+-----------+
| John       | Doe       | 6000.00 |  900.0000 |
| Jane       | Smith     | 7000.00 | 1050.0000 |
| Sam        | Adams     | 9000.00 | 1350.0000 |
| Emily      | Clark     | 5000.00 |  750.0000 |
| Daniel     | Harris    | 3000.00 |  450.0000 |
| Rachel     | Baker     | 8000.00 | 1200.0000 |
| Paul       | Jones     | 5500.00 |  825.0000 |
| Sophia     | Taylor    | 6500.00 |  975.0000 |
| Mike       | Brown     | 4000.00 |  600.0000 |
+------------+-----------+---------+-----------+
```

11. Write a query to select all record from employees where last name in 'BLAKE', 'SCOTT', 'KING' and 'FORD'.

```
mysql> SELECT *
    -> FROM employees
    -> WHERE last_name IN ('BLAKE', 'SCOTT', 'KING', 'FORD');
Empty set (0.00 sec)
```

MySQL Sub Queries Exercises

1. Write a query to find the names (first_name, last_name) and the salaries of the employees who have a higher salary than the employee whose last_name='Bull'.

```
mysql> SELECT
    ->     first_name,
    ->     last_name,
    ->     salary
    -> FROM employees
    -> WHERE salary > (
    ->     SELECT salary
    ->     FROM employees
    ->     WHERE last_name = 'Bull'
    -> );
Empty set (0.00 sec)
```

2. Write a query to find the names (first_name, last_name) of all employees who works in the IT department.

```
mysql> SELECT
    ->     e.first_name,
    ->     e.last_name
    -> FROM employees e
    -> JOIN departments d
    -> ON e.department_id = d.department_id
    -> WHERE d.department_name = 'IT';
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| John       | Doe       |
| Sam        | Adams     |
+------------+-----------+
2 rows in set (0.00 sec)
```

3. Write a query to find the names (first_name, last_name) of the employees who have a manager and work for a department based in the United States.
Hint : Write single-row and multiple-row subqueries
Sample table: employees
Sample table: departments
Sample table: locations

```
mysql> SELECT first_name, last_name
    -> FROM employees
    -> WHERE manager_id IN(select employee_id FROM employees WHERE departmen
t_id
    -> IN (SELECT department_id FROM departments WHERE location_id
    -> IN (select location_id from locations where country_id='US')));
Empty set (0.00 sec)
```

4. Write a query to find the names (first_name, last_name) of the employees who are managers.

```
mysql> SELECT first_name, last_name
    -> FROM employees
    -> WHERE (employee_id IN (SELECT manager_id FROM employees));
Empty set (0.00 sec)
```

5. Write a query to find the names (first_name, last_name), the salary of the employees whose salary is greater than the average salary.

```
mysql> SELECT first_name, last_name, salary
    -> FROM employees
    -> WHERE salary > (SELECT AVG(salary) FROM employees);
+------------+-----------+---------+
| first_name | last_name | salary  |
+------------+-----------+---------+
| Jane       | Smith     | 7000.00 |
| Sam        | Adams     | 9000.00 |
| Rachel     | Baker     | 8000.00 |
| Sophia     | Taylor    | 6500.00 |
+------------+-----------+---------+
4 rows in set (0.00 sec)
```

6. Write a query to find the names (first_name, last_name), the salary of the employees whose salary is equal to the minimum salary for their job grade.
Sample table: employees
Sample table: jobs

```
mysql> SELECT first_name, last_name, salary
    -> FROM employees
    -> WHERE employees.salary = (SELECT min_salary FROM jobs
    ->    WHERE employees.job_id = jobs.job_id);
+------------+-----------+---------+
| first_name | last_name | salary  |
+------------+-----------+---------+
| Mike       | Brown     | 4000.00 |
+------------+-----------+---------+
1 row in set (0.00 sec)
```

7. Write a query to find the names (first_name, last_name), the salary of the employees who earn more than the average salary and who works in any of the IT departments.
Sample table: employees
Sample table: departments

```
mysql> SELECT first_name, last_name, salary
    -> FROM employees
    -> WHERE department_id IN (SELECT department_id FROM departments
    ->    WHERE department_name LIKE 'IT%') AND salary > (SELECT avg(salary)
 FROM employees);
+------------+-----------+---------+
| first_name | last_name | salary  |
+------------+-----------+---------+
| Sam        | Adams     | 9000.00 |
+------------+-----------+---------+
1 row in set (0.00 sec)
```

8. Write a query to find the names (first_name, last_name), the salary of the employees who earn more than Mr. Bell.
Sample table: employees
Sample table: departments

```
mysql> SELECT first_name, last_name, salary
    -> FROM employees
    -> WHERE salary >
    -> (SELECT salary FROM employees WHERE last_name = 'Bell') ORDER BY firs
t_name;
Empty set (0.00 sec)
```

9. Write a query to find the names (first_name, last_name), the salary of the employees who earn the same salary as the minimum salary for all departments.
Sample table: employees
Sample table: departments

```
mysql> SELECT *
    -> FROM employees
    -> WHERE salary = (SELECT MIN(salary) FROM employees);
+-------------+------------+-----------+--------+--------------+------------+----------+---------+----------------+------------+---------------+
| employee_id | first_name | last_name | email  | phone_number | hire_date  | job_id   | salary  | commission_pct | manager_id | department_id |
+-------------+------------+-----------+--------+--------------+------------+----------+---------+----------------+------------+---------------+
|         105 | Daniel     | Harris    | DHARRIS | 123460       | 2016-07-19 | SH_CLERK | 3000.00 |           NULL |       NULL |            50 |
+-------------+------------+-----------+--------+--------------+------------+----------+---------+----------------+------------+---------------+
1 row in set (0.00 sec)
```

10. Write a query to find the names (first_name, last_name), the salary of the employees whose salary greater than the average salary of all departments.
Sample table: employees

```
mysql> SELECT *
    -> FROM employees
    -> WHERE salary >
    -> ALL(SELECT avg(salary)
    -> FROM employees GROUP BY department_id);
+-------------+------------+-----------+--------+--------------+------------+---------+---------+----------------+------------+---------------+
| employee_id | first_name | last_name | email  | phone_number | hire_date  | job_id  | salary  | commission_pct | manager_id | department_id |
+-------------+------------+-----------+--------+--------------+------------+---------+---------+----------------+------------+---------------+
|         103 | Sam        | Adams     | SADAMS | 123458       | 2012-11-04 | IT_PROG | 9000.00 |           NULL |       NULL |            60 |
|         106 | Rachel     | Baker     | RBAKER | 123461       | 2019-10-01 | SA_REP  | 8000.00 |           0.20 |       NULL |            80 |
+-------------+------------+-----------+--------+--------------+------------+---------+---------+----------------+------------+---------------+
2 rows in set (0.00 sec)
```

11. Write a query to find the names (first_name, last_name) and salary of the employees who earn a salary that is higher than the salary of all the Shipping Clerk (JOB_ID = 'SH_CLERK'). Sort the results of the salary of the lowest to highest.
Sample table: employees

```
mysql> SELECT first_name,last_name, job_id, salary
    -> FROM employees
    -> WHERE salary >
    -> ALL (SELECT salary FROM employees WHERE job_id = 'SH_CLERK')
    -> ORDER BY salary ;
+------------+-----------+---------+---------+
| first_name | last_name | job_id  | salary  |
+------------+-----------+---------+---------+
| Mike       | Brown     | IT_PROG | 4000.00 |
| Emily      | Clark     | HR_REP  | 5000.00 |
| Paul       | Jones     | SA_REP  | 5500.00 |
| John       | Doe       | IT_PROG | 6000.00 |
| Sophia     | Taylor    | HR_REP  | 6500.00 |
| Jane       | Smith     | SA_REP  | 7000.00 |
| Rachel     | Baker     | SA_REP  | 8000.00 |
| Sam        | Adams     | IT_PROG | 9000.00 |
+------------+-----------+---------+---------+
8 rows in set (0.00 sec)
```

12. Write a query to find the names (first_name, last_name) of the employees who are not supervisors.
Sample table: employees

```
mysql> SELECT b.first_name,b.last_name
    -> FROM employees b
    -> WHERE NOT EXISTS
    -> (SELECT 'X' FROM employees a WHERE a.manager_id = b.employee_id);
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| John       | Doe       |
| Jane       | Smith     |
| Sam        | Adams     |
| Emily      | Clark     |
| Daniel     | Harris    |
| Rachel     | Baker     |
| Paul       | Jones     |
| Sophia     | Taylor    |
| Mike       | Brown     |
+------------+-----------+
9 rows in set (0.00 sec)
```

13. Write a query to display the employee ID, first name, last names, and department names of all employees.
Sample table: employees
Sample table: departments

```
mysql> SELECT employee_id, first_name, last_name,
    -> (SELECT department_name FROM departments d
    ->  WHERE e.department_id = d.department_id) department
    ->  FROM employees e ORDER BY department;
+-------------+------------+-----------+-----------------+
| employee_id | first_name | last_name | department      |
+-------------+------------+-----------+-----------------+
|         104 | Emily      | Clark     | Human Resources |
|         108 | Sophia     | Taylor    | Human Resources |
|         101 | John       | Doe       | IT              |
|         103 | Sam        | Adams     | IT              |
|         118 | Mike       | Brown     | Purchasing      |
|         102 | Jane       | Smith     | Sales           |
|         106 | Rachel     | Baker     | Sales           |
|         107 | Paul       | Jones     | Sales           |
|         105 | Daniel     | Harris    | Shipping        |
+-------------+------------+-----------+-----------------+
9 rows in set (0.01 sec)
```

14. Write a query to display the employee ID, first name, last names, salary of all employees whose salary is above average for their departments.
Sample table: employees
Sample table: departments

```
mysql> SELECT employee_id, first_name
    -> FROM employees
    -> AS A WHERE salary >
    -> ( SELECT AVG(salary) FROM employees WHERE department_id = A.department_i
+-------------+------------+
| employee_id | first_name |
+-------------+------------+
|         102 | Jane       |
|         103 | Sam        |
|         106 | Rachel     |
|         108 | Sophia     |
+-------------+------------+
4 rows in set (0.00 sec)
```

15. Write a query to fetch even numbered records from employees table.
Sample table: employees

```
mysql> SET @i = 0;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT i, employee_id
    -> FROM (SELECT @i := @i + 1 AS i, employee_id FROM employees)
    -> a WHERE MOD(a.i, 2) = 0;
+------+-------------+
| i    | employee_id |
+------+-------------+
|    2 |         102 |
|    4 |         104 |
|    6 |         106 |
|    8 |         108 |
+------+-------------+
4 rows in set (0.00 sec)
```

16. Write a query to find the 5th maximum salary in the employees table.
Sample table: employees

```
mysql> SELECT DISTINCT salary
    -> FROM employees e1
    -> WHERE 5 = (SELECT COUNT(DISTINCT salary)
    -> FROM employees  e2
    ->  WHERE e2.salary >= e1.salary);
+---------+
| salary  |
+---------+
| 6000.00 |
+---------+
1 row in set (0.00 sec)
```

17. Write a query to find the 4th minimum salary in the employees table.
Sample table: employees

```
mysql> SELECT DISTINCT salary
    -> FROM employees e1
    -> WHERE 4 = (SELECT COUNT(DISTINCT salary)
    -> FROM employees  e2 WHERE e2.salary <= e1.salary);
+---------+
| salary  |
+---------+
| 5500.00 |
+---------+
1 row in set (0.00 sec)
```

18. Write a query to select last 10 records from a table.
Sample table: employees

```
mysql> SELECT * FROM
   -> (SELECT * FROM employees
   -> ORDER BY employee_id DESC LIMIT 10)
   -> sub ORDER BY employee_id ASC;
+-------------+------------+-----------+---------+--------------+------------+----------+---------+----------------+------------+--------------+
| employee_id | first_name | last_name | email   | phone_number | hire_date  | job_id   | salary  | commission_pct | manager_id | department_id |
+-------------+------------+-----------+---------+--------------+------------+----------+---------+----------------+------------+--------------+
|         101 | John       | Doe       | JDOE    | 123456       | 2015-06-23 | IT_PROG  | 6000.00 |           NULL |       NULL |            60 |
|         102 | Jane       | Smith     | JSMITH  | 123457       | 2018-02-10 | SA_REP   | 7000.00 |           0.15 |       NULL |            80 |
|         103 | Sam        | Adams     | SADAMS  | 123458       | 2012-11-04 | IT_PROG  | 9000.00 |           NULL |       NULL |            60 |
|         104 | Emily      | Clark     | ECLARK  | 123459       | 2020-03-15 | HR_REP   | 5000.00 |           NULL |       NULL |            40 |
|         105 | Daniel     | Harris    | DHARRIS | 123460       | 2016-07-19 | SH_CLERK | 3000.00 |           NULL |       NULL |            50 |
|         106 | Rachel     | Baker     | RBAKER  | 123461       | 2019-10-01 | SA_REP   | 8000.00 |           0.20 |       NULL |            80 |
|         107 | Paul       | Jones     | PJONES  | 123462       | 2017-09-13 | SA_REP   | 5500.00 |           0.10 |       NULL |            80 |
|         108 | Sophia     | Taylor    | STAYLOR | 123463       | 2014-12-21 | HR_REP   | 6500.00 |           NULL |       NULL |            40 |
|         118 | Mike       | Brown     | MBROWN  | 123470       | 2013-05-10 | IT_PROG  | 4000.00 |           NULL |       NULL |            30 |
+-------------+------------+-----------+---------+--------------+------------+----------+---------+----------------+------------+--------------+
9 rows in set (0.00 sec)
```

19. Write a query to list department number, name for all the departments in which there are no employees in the department.
Sample table: employees
Sample table: departments

```
mysql> SELECT *
    -> FROM departments
    -> WHERE department_id
    -> NOT IN (select department_id FROM employees);
+---------------+-----------------+------------+-------------+
| department_id | department_name | manager_id | location_id |
+---------------+-----------------+------------+-------------+
|            10 | Administration  |       NULL |        NULL |
|            20 | Marketing       |       NULL |        NULL |
|            90 | Executive       |       NULL |        NULL |
+---------------+-----------------+------------+-------------+
3 rows in set (0.00 sec)
```

20. Write a query to get 3 maximum salaries.

```
mysql> SELECT DISTINCT salary
    -> FROM employees a
    -> WHERE 3 >= (SELECT COUNT(DISTINCT salary)
    -> FROM employees b
    -> WHERE b.salary <= a.salary) ORDER BY a.salary DESC;
+---------+
| salary  |
+---------+
| 5000.00 |
| 4000.00 |
| 3000.00 |
+---------+
3 rows in set (0.00 sec)
```

21. Write a query to get 3 minimum salaries.

```
mysql> SELECT DISTINCT salary
    -> FROM employees a
    -> WHERE  3 >= (SELECT COUNT(DISTINCT salary)
    -> FROM employees b WHERE b.salary >= a.salary)
    -> ORDER BY a.salary DESC;
+---------+
| salary  |
+---------+
| 9000.00 |
| 8000.00 |
| 7000.00 |
+---------+
3 rows in set (0.00 sec)
```

22. Write a query to get nth max salaries of employees.

```
mysql> SELECT * FROM employees emp1 WHERE (1) = (SELECT COUNT(DISTINCT(emp2.salary)) FROM employees emp2 WHERE emp2.salary > emp1.salary);
+-------------+------------+-----------+--------+--------------+------------+--------+---------+----------------+------------+---------------+
| employee_id | first_name | last_name | email  | phone_number | hire_date  | job_id | salary  | commission_pct | manager_id | department_id |
+-------------+------------+-----------+--------+--------------+------------+--------+---------+----------------+------------+---------------+
|         106 | Rachel     | Baker     | RBAKER | 123461       | 2019-10-01 | SA_REP | 8000.00 |           0.20 |       NULL |            80 |
+-------------+------------+-----------+--------+--------------+------------+--------+---------+----------------+------------+---------------+
```

MySQL Sub Queries Solutions

1
SELECT FIRST_NAME, LAST_NAME, SALARY
FROM employees
WHERE SALARY > (SELECT salary FROM employees WHERE last_name = 'Bull');
2
SELECT first_name, last_name

```
FROM employees
WHERE department_id
IN (SELECT department_id FROM departments WHERE department_name='IT');
```

3

```
SELECT first_name, last_name
FROM employees
WHERE manager_id IN(select employee_id FROM employees WHERE department_id
IN (SELECT department_id FROM departments WHERE location_id
IN (select location_id from locations where country_id='US')));
```

4

```
SELECT first_name, last_name
FROM employees
WHERE (employee_id IN (SELECT manager_id FROM employees));
```

5

```
SELECT first_name, last_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

6

```
SELECT first_name, last_name, salary
FROM employees
WHERE employees.salary = (SELECT min_salary FROM jobs
  WHERE employees.job_id = jobs.job_id);
```

7

```
SELECT first_name, last_name, salary
FROM employees
WHERE department_id IN (SELECT department_id FROM departments
  WHERE department_name LIKE 'IT%') AND salary > (SELECT avg(salary) FROM
employees);
```

8

```
SELECT first_name, last_name, salary
FROM employees
WHERE salary >
(SELECT salary FROM employees WHERE last_name = 'Bell') ORDER BY first_name;
```

9

```
SELECT *
FROM employees
WHERE salary = (SELECT MIN(salary) FROM employees);
```

10

```
SELECT *
FROM employees
```

```
WHERE salary >
ALL(SELECT avg(salary)
FROM employees GROUP BY department_id);
```

11.

```
SELECT first_name,last_name, job_id, salary
FROM employees
WHERE salary >
ALL (SELECT salary FROM employees WHERE job_id = 'SH_CLERK')
ORDER BY salary ;
```

12.

```
SELECT b.first_name,b.last_name
FROM employees b
WHERE NOT EXISTS
(SELECT 'X' FROM employees a WHERE a.manager_id = b.employee_id);
```

13

```
SELECT employee_id, first_name, last_name,
(SELECT department_name FROM departments d
 WHERE e.department_id = d.department_id) department
 FROM employees e ORDER BY department;
```

14

```
SELECT employee_id, first_name
FROM employees
AS A WHERE salary >
( SELECT AVG(salary) FROM employees WHERE department_id = A.department_id);
```

15

```
SET @i = 0;
SELECT i, employee_id
FROM (SELECT @i := @i + 1 AS i, employee_id FROM employees)
a WHERE MOD(a.i, 2) = 0;
```

16

```
SELECT DISTINCT salary
FROM employees e1
WHERE 5 = (SELECT COUNT(DISTINCT salary)
FROM employees  e2
 WHERE e2.salary >= e1.salary);
```

17

```
SELECT DISTINCT salary
FROM employees e1
WHERE 4 = (SELECT COUNT(DISTINCT salary)
FROM employees  e2 WHERE e2.salary <= e1.salary);
```

18

```
SELECT * FROM
```

```
(SELECT * FROM employees
ORDER BY employee_id DESC LIMIT 10)
sub ORDER BY employee_id ASC;
```

19

```
SELECT *
FROM departments
WHERE department_id
NOT IN (select department_id FROM employees);
```

20

```
SELECT DISTINCT salary
FROM employees a
WHERE 3 >= (SELECT COUNT(DISTINCT salary)
FROM employees b
WHERE b.salary <= a.salary) ORDER BY a.salary DESC;
```

21

```
SELECT DISTINCT salary
FROM employees a
WHERE  3 >= (SELECT COUNT(DISTINCT salary)
FROM employees b WHERE b.salary >= a.salary)
ORDER BY a.salary DESC;
```

22

```
SELECT * FROM employees emp1 WHERE (1) = (SELECT
COUNT(DISTINCT(emp2.salary)) FROM employees emp2 WHERE emp2.salary >
emp1.salary);
```

MySQL JOINS – Exercises

1. Write a query to find the addresses (location_id, street_address, city, state_province, country_name) of all the departments.
Hint : Use NATURAL JOIN.
Sample table : locations
Sample table : countries

```
mysql> SELECT
    ->     location_id,
    ->     street_address,
    ->     state,
    ->     country_name
    -> FROM locations
    -> NATURAL JOIN country_new;
Empty set (0.01 sec)
```

2. Write a query to find the names (first_name, last name), department ID and name of all the employees.
Sample table : employees
Sample table : departments

```
mysql> SELECT
    ->     e.first_name,
    ->     e.last_name,
    ->     e.department_id,
    ->     d.department_name
    -> FROM employees e
    -> JOIN departments d
    -> ON e.department_id = d.department_id;
+------------+-----------+---------------+-----------------+
| first_name | last_name | department_id | department_name |
+------------+-----------+---------------+-----------------+
| John       | Doe       |            60 | IT              |
| Jane       | Smith     |            80 | Sales           |
| Sam        | Adams     |            60 | IT              |
| Emily      | Clark     |            40 | Human Resources |
| Daniel     | Harris    |            50 | Shipping        |
| Rachel     | Baker     |            80 | Sales           |
| Paul       | Jones     |            80 | Sales           |
| Sophia     | Taylor    |            40 | Human Resources |
| Mike       | Brown     |            30 | Purchasing      |
+------------+-----------+---------------+-----------------+
9 rows in set (0.00 sec)
```

3. Find the names (first_name, last_name), job, department number, and department name of the employees who work in London.
Sample table : departments
Sample table : locations

```
mysql> SELECT
    ->     e.first_name,
    ->     e.last_name,
    ->     j.job_title,
    ->     e.department_id,
    ->     d.department_name
    -> FROM employees e
    -> JOIN departments d
    -> ON e.department_id = d.department_id
    -> JOIN locations l
    -> ON d.location_id = l.location_id
    -> JOIN jobs j
    -> ON e.job_id = j.job_id
    -> WHERE l.state = 'London';
Empty set (0.00 sec)
```

4. Write a query to find the employee id, name (last_name) along with their manager_id, manager name (last_name).
Sample table : employees

```
mysql> SELECT
    ->     e.employee_id,
    ->     e.last_name AS employee_name,
    ->     e.manager_id,
    ->     m.last_name AS manager_name
    -> FROM employees e
    -> LEFT JOIN employees m
    -> ON e.manager_id = m.employee_id;
+-------------+---------------+------------+--------------+
| employee_id | employee_name | manager_id | manager_name |
+-------------+---------------+------------+--------------+
|         101 | Doe           |       NULL | NULL         |
|         102 | Smith         |       NULL | NULL         |
|         103 | Adams         |       NULL | NULL         |
|         104 | Clark         |       NULL | NULL         |
|         105 | Harris        |       NULL | NULL         |
|         106 | Baker         |       NULL | NULL         |
|         107 | Jones         |       NULL | NULL         |
|         108 | Taylor        |       NULL | NULL         |
|         118 | Brown         |       NULL | NULL         |
+-------------+---------------+------------+--------------+
9 rows in set (0.00 sec)
```

5. Find the names (first_name, last_name) and hire date of the employees who were hired after 'Jones'.
Sample table : employees

```
mysql> SELECT
    ->     first_name,
    ->     last_name,
    ->     hire_date
    -> FROM employees
    -> WHERE hire_date > (
    ->     SELECT hire_date
    ->     FROM employees
    ->     WHERE last_name = 'Jones'
    -> );
+------------+-----------+------------+
| first_name | last_name | hire_date  |
+------------+-----------+------------+
| Jane       | Smith     | 2018-02-10 |
| Emily      | Clark     | 2020-03-15 |
| Rachel     | Baker     | 2019-10-01 |
+------------+-----------+------------+
3 rows in set (0.00 sec)
```

6. Write a query to get the department name and number of employees in the department.
Sample table : employees
Sample table : departments

```
mysql> SELECT
    ->     d.department_name,
    ->     COUNT(e.employee_id) AS employee_count
    -> FROM departments d
    -> LEFT JOIN employees e
    -> ON d.department_id = e.department_id
    -> GROUP BY d.department_name;
+-----------------+----------------+
| department_name | employee_count |
+-----------------+----------------+
| Administration  |              0 |
| Executive       |              0 |
| Human Resources |              2 |
| IT              |              2 |
| Marketing       |              0 |
| Purchasing      |              1 |
| Sales           |              3 |
| Shipping        |              1 |
+-----------------+----------------+
8 rows in set (0.00 sec)
```

7. Find the employee ID, job title, number of days between ending date and starting date for all jobs in department 90 from job history.
Sample table : employees

```
mysql> SELECT
    ->     employee_id,
    ->     job_id,
    ->     DATEDIFF(end_date, start_date) AS days_worked
    -> FROM job_history
    -> WHERE department_id = 90;
Empty set (0.01 sec)
```

8. Write a query to display the department ID, department name and manager first name.
Sample table : employees
Sample table : departments

```
mysql> SELECT
    ->     d.department_id,
    ->     d.department_name,
    ->     e.first_name AS manager_first_name
    -> FROM departments d
    -> LEFT JOIN employees e
    -> ON d.manager_id = e.employee_id;
+---------------+-----------------+-------------------+
| department_id | department_name | manager_first_name |
+---------------+-----------------+-------------------+
|            10 | Administration  | NULL              |
|            20 | Marketing       | NULL              |
|            30 | Purchasing      | NULL              |
|            40 | Human Resources | NULL              |
|            50 | Shipping        | NULL              |
|            60 | IT              | NULL              |
|            80 | Sales           | NULL              |
|            90 | Executive       | NULL              |
+---------------+-----------------+-------------------+
8 rows in set (0.00 sec)
```

9. Write a query to display the department name, manager name, and city.
Sample table : employees
Sample table : departments
Sample table : locations

```
mysql> SELECT
    ->     d.department_name,
    ->     e.first_name AS manager_name,
    ->     l.state AS city
    -> FROM departments d
    -> LEFT JOIN employees e
    -> ON d.manager_id = e.employee_id
    -> LEFT JOIN locations l
    -> ON d.location_id = l.location_id;
+-----------------+--------------+------+
| department_name | manager_name | city |
+-----------------+--------------+------+
| Administration  | NULL         | NULL |
| Marketing       | NULL         | NULL |
| Purchasing      | NULL         | NULL |
| Human Resources | NULL         | NULL |
| Shipping        | NULL         | NULL |
| IT              | NULL         | NULL |
| Sales           | NULL         | NULL |
| Executive       | NULL         | NULL |
+-----------------+--------------+------+
8 rows in set (0.00 sec)
```

10. Write a query to display the job title and average salary of employees.
Sample table : employees

```
mysql> SELECT
    ->     j.job_title,
    ->     AVG(e.salary) AS avg_salary
    -> FROM employees e
    -> JOIN jobs j
    -> ON e.job_id = j.job_id
    -> GROUP BY j.job_title;
+----------------------+-------------+
| job_title            | avg_salary  |
+----------------------+-------------+
| HR Representative    | 5750.000000 |
| Programmer           | 6333.333333 |
| Sales Representative | 6833.333333 |
| Shipping Clerk       | 3000.000000 |
+----------------------+-------------+
4 rows in set (0.01 sec)
```

11. Display job title, employee name, and the difference between salary of the employee and minimum salary for the job.
Sample table : employees

```
mysql> SELECT
    ->     j.job_title,
    ->     CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
    ->     e.salary,
    ->     j.min_salary,
    ->     (e.salary - j.min_salary) AS salary_difference
    -> FROM employees e
    -> JOIN jobs j
    -> ON e.job_id = j.job_id;
+----------------------+---------------+---------+------------+------------------+
| job_title            | employee_name | salary  | min_salary | salary_difference |
+----------------------+---------------+---------+------------+------------------+
| HR Representative    | Emily Clark   | 5000.00 |       4000 |          1000.00 |
| HR Representative    | Sophia Taylor | 6500.00 |       4000 |          2500.00 |
| Programmer           | John Doe      | 6000.00 |       4000 |          2000.00 |
| Programmer           | Sam Adams     | 9000.00 |       4000 |          5000.00 |
| Programmer           | Mike Brown    | 4000.00 |       4000 |             0.00 |
| Sales Representative | Jane Smith    | 7000.00 |       3000 |          4000.00 |
| Sales Representative | Rachel Baker  | 8000.00 |       3000 |          5000.00 |
| Sales Representative | Paul Jones    | 5500.00 |       3000 |          2500.00 |
| Shipping Clerk       | Daniel Harris | 3000.00 |       2500 |           500.00 |
+----------------------+---------------+---------+------------+------------------+
9 rows in set (0.00 sec)
```

12. Write a query to display the job history that were done by any employee who is currently drawing more than 10000 of salary.
Sample table : employees
Sample table : Job_history

```
mysql> SELECT
    ->     jh.employee_id,
    ->     jh.start_date,
    ->     jh.end_date,
    ->     jh.job_id,
    ->     jh.department_id
    -> FROM job_history jh
    -> JOIN employees e
    -> ON jh.employee_id = e.employee_id
    -> WHERE e.salary > 10000;
Empty set (0.00 sec)
```

13. Write a query to display department name, name (first_name, last_name), hire date, salary of the manager for all managers whose experience is more than 15 years.
Sample table : employees
Sample table : departments

```
mysql> SELECT
    ->     d.department_name,
    ->     e.first_name,
    ->     e.last_name,
    ->     e.hire_date,
    ->     e.salary
    -> FROM departments d
    -> JOIN employees e
    -> ON d.manager_id = e.employee_id
    -> WHERE TIMESTAMPDIFF(YEAR, e.hire_date, CURDATE()) > 15;
Empty set (0.00 sec)
```

Joins Exercises Solutions

1
```
SELECT location_id, street_address, city, state_province, country_name
FROM locations
NATURAL JOIN countries;
```
2
```
SELECT first_name, last_name, department_id, department_name
FROM employees
JOIN
departments
USING (department_id);
```
3
```
SELECT e.first_name, e.last_name, e.job_id, e.department_id, d.department_name
FROM employees e
 JOIN
 departments d ON (e.department_id = d.department_id)
 JOIN
 locations l ON (d.location_id = l.location_id)
 WHERE LOWER(l.city) = 'London';
```
4
```
SELECT e.employee_id 'Emp_Id', e.last_name 'Employee', m.employee_id 'Mgr_Id',
m.last_name 'Manager'
FROM employees e
join employees m
 ON (e.manager_id = m.employee_id);
```
5
```
SELECT e.first_name, e.last_name, e.hire_date
 FROM employees e
 JOIN
 employees davies
 ON (davies.last_name = 'Jones')
 WHERE davies.hire_date < e.hire_date;
```

6.

```
SELECT department_name AS 'Department Name',
COUNT(*) AS 'No of Employees'
FROM departments
INNER JOIN employees
ON employees.department_id = departments.department_id
GROUP BY departments.department_id, department_name
ORDER BY department_name;
```

7

```
SELECT employee_id, job_title, end_date-start_date Days FROM job_history
NATURAL JOIN jobs
WHERE department_id=90;
```

8

```
SELECT d.department_id, d.department_name, e.manager_id, e.first_name
FROM departments d
INNER JOIN employees e
ON (d.manager_id = e.employee_id);
```

9

```
SELECT d.department_name, e.first_name, l.city
FROM departments d
JOIN employees e
ON (d.manager_id = e.employee_id)
JOIN locations l USING (location_id);
```

10

```
SELECT job_title, AVG(salary)
FROM employees
NATURAL JOIN jobs
GROUP BY job_title;
```

11

```
SELECT job_title, first_name, salary-min_salary 'Salary - Min_Salary'
FROM employees
NATURAL JOIN jobs;
```

12

```
SELECT jh.* FROM job_history jh
JOIN employees e
ON (jh.employee_id = e.employee_id)
WHERE salary > 10000;
```

13.

```
SELECT first_name, last_name, hire_date, salary,
(DATEDIFF(now(), hire_date))/365 Experience
FROM departments d JOIN employees e
ON (d.manager_id = e.employee_id)
WHERE (DATEDIFF(now(), hire_date))/365>15;
```

MySQL Northwind database, Products table – Exercises

1. Write a query to get Product name and quantity/unit.

```
mysql> SELECT
    ->     ProductName,
    ->     QuantityPerUnit
    -> FROM Products;
Empty set (0.00 sec)
```

2. Write a query to get current Product list (Product ID and name).

```
mysql> SELECT
    ->     ProductID,
    ->     ProductName
    -> FROM Products
    -> WHERE Discontinued = 0;
Empty set (0.00 sec)
```

3. Write a query to get discontinued Product list (Product ID and name).

```
mysql> SELECT
    ->     ProductID,
    ->     ProductName
    -> FROM Products
    -> WHERE Discontinued = 1;
Empty set (0.00 sec)
```

4. Write a query to get most expense and least expensive Product list (name and unit price).

```
mysql> SELECT
    ->     ProductName,
    ->     UnitPrice
    -> FROM Products
    -> WHERE UnitPrice = (SELECT MAX(UnitPrice) FROM Products)
    ->    OR UnitPrice = (SELECT MIN(UnitPrice) FROM Products);
Empty set (0.00 sec)
```

5. Write a query to get Product list (id, name, unit price) where current products cost less than $20.

```
mysql> SELECT
    ->     ProductID,
    ->     ProductName,
    ->     UnitPrice
    -> FROM Products
    -> WHERE UnitPrice < 20
    -> AND Discontinued = 0;
Empty set (0.00 sec)
```

6. Write a query to get Product list (id, name, unit price) where products cost between $15 and $25.

```
mysql> SELECT
    ->     ProductID,
    ->     ProductName,
    ->     UnitPrice
    -> FROM Products
    -> WHERE UnitPrice BETWEEN 15 AND 25;
Empty set (0.00 sec)
```

7. Write a query to get Product list (name, unit price) of above average price.

```
mysql> SELECT
    ->     ProductName,
    ->     UnitPrice
    -> FROM Products
    -> WHERE UnitPrice > (
    ->     SELECT AVG(UnitPrice)
    ->     FROM Products
    -> );
Empty set (0.00 sec)
```

8. Write a query to get Product list (name, unit price) of ten most expensive products.

```
mysql> SELECT
    ->     ProductName,
    ->     UnitPrice
    -> FROM Products
    -> ORDER BY UnitPrice DESC
    -> LIMIT 10;
Empty set (0.00 sec)
```

9. Write a query to count current and discontinued products.

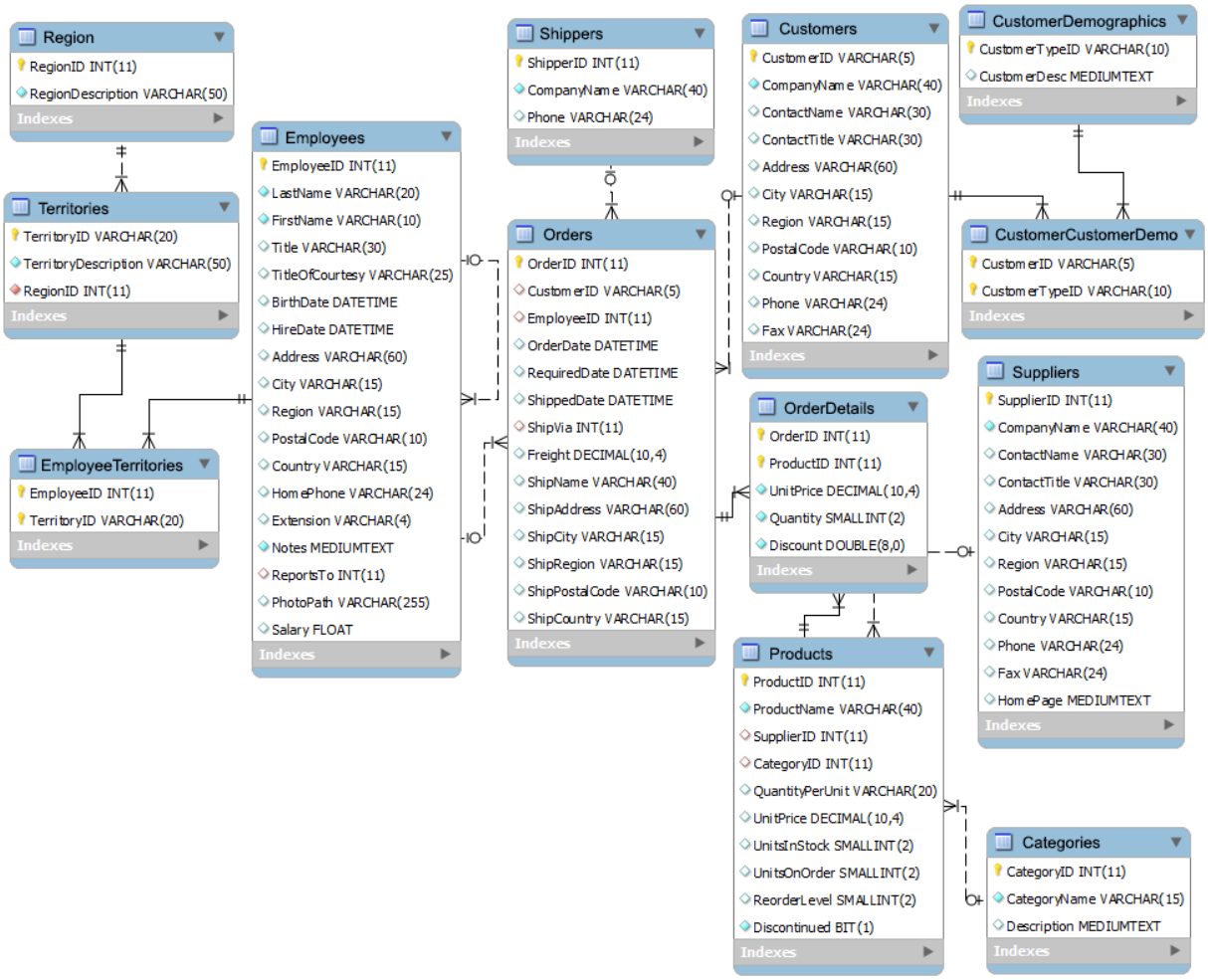
```
mysql> SELECT
    ->     Discontinued,
    ->     COUNT(*) AS product_count
    -> FROM Products
    -> GROUP BY Discontinued;
Empty set (0.00 sec)
```

10. Write a query to get Product list (name, units on order , units in stock) of stock is less than the quantity on order.

```
mysql> SELECT
    ->     ProductName,
    ->     UnitsOnOrder,
    ->     UnitsInStock
    -> FROM Products
    -> WHERE UnitsInStock < UnitsOnOrder;
Empty set (0.00 sec)
```

... More

Structure of 'northwind' database :


Region
RegionID INT(11)
RegionDescription VARCHAR(50)
Indexes
Territories
TerritoryID VARCHAR(20)
TerritoryDescription VARCHAR(50)
RegionID INT(11)
Indexes
EmployeeTerritories
EmployeeID INT(11)
TerritoryID VARCHAR(20)
Indexes
Employees
EmployeeID INT(11)
LastName VARCHAR(20)
FirstName VARCHAR(10)
Title VARCHAR(30)
TitleOfCourtesy VARCHAR(25)
BirthDate DATETIME
HireDate DATETIME
Address VARCHAR(60)
City VARCHAR(15)
Region VARCHAR(15)
PostalCode VARCHAR(10)
Country VARCHAR(15)
HomePhone VARCHAR(24)
Extension VARCHAR(4)
Notes MEDIUMTEXT
ReportsTo INT(11)
PhotoPath VARCHAR(255)
Salary FLOAT
Indexes
Shippers
ShipperID INT(11)
CompanyName VARCHAR(40)
Phone VARCHAR(24)
Indexes
Orders
OrderID INT(11)
CustomerID VARCHAR(5)
EmployeeID INT(11)
OrderDate DATETIME
RequiredDate DATETIME
ShippedDate DATETIME
ShipVia INT(11)
Freight DECIMAL(10,4)
ShipName VARCHAR(40)
ShipAddress VARCHAR(60)
ShipCity VARCHAR(15)
ShipRegion VARCHAR(15)
ShipPostalCode VARCHAR(10)
ShipCountry VARCHAR(15)
Indexes
Customers
CustomerID VARCHAR(5)
CompanyName VARCHAR(40)
ContactName VARCHAR(30)
ContactTitle VARCHAR(30)
Address VARCHAR(60)
City VARCHAR(15)
Region VARCHAR(15)
PostalCode VARCHAR(10)
Country VARCHAR(15)
Phone VARCHAR(24)
Fax VARCHAR(24)
Indexes
CustomerDemographics
CustomerTypeID VARCHAR(10)
CustomerDesc MEDIUMTEXT
Indexes
CustomerCustomerDemo
CustomerID VARCHAR(5)
CustomerTypeID VARCHAR(10)
Indexes
OrderDetails
OrderID INT(11)
ProductID INT(11)
UnitPrice DECIMAL(10,4)
Quantity SMALLINT(2)
Discount DOUBLE(8,0)
Indexes
Suppliers
SupplierID INT(11)
CompanyName VARCHAR(40)
ContactName VARCHAR(30)
ContactTitle VARCHAR(30)
Address VARCHAR(60)
City VARCHAR(15)
Region VARCHAR(15)
PostalCode VARCHAR(10)
Country VARCHAR(15)
Phone VARCHAR(24)
Fax VARCHAR(24)
HomePage MEDIUMTEXT
Indexes
Products
ProductID INT(11)
ProductName VARCHAR(40)
SupplierID INT(11)
CategoryID INT(11)
QuantityPerUnit VARCHAR(20)
UnitPrice DECIMAL(10,4)
UnitsInStock SMALLINT(2)
UnitsOnOrder SMALLINT(2)
ReorderLevel SMALLINT(2)
Discontinued BIT(1)
Indexes
Categories
CategoryID INT(11)
CategoryName VARCHAR(15)
Description MEDIUMTEXT
Indexes

MySQL Northwind database, Products table – Solution

1
```
SELECT ProductName, QuantityPerUnit
FROM Products;
```
2
```
SELECT ProductID, ProductName
FROM Products
WHERE Discontinued = "False"
ORDER BY ProductName;
```
3
```
SELECT ProductID, ProductName
FROM Products
WHERE Discontinued = "True"
ORDER BY ProductName;
```
4
```
SELECT ProductName, UnitPrice
FROM Products
ORDER BY UnitPrice DESC;
```
5
```
SELECT ProductID, ProductName, UnitPrice
FROM Products
WHERE (((UnitPrice)<20) AND ((Discontinued)=False))
ORDER BY UnitPrice DESC;
```
6
```
SELECT ProductName, UnitPrice
FROM Products
WHERE (((UnitPrice)>=15 And (UnitPrice)<=25)
AND ((Products.Discontinued)=False))
ORDER BY Products.UnitPrice DESC;
```
7
```
SELECT DISTINCT ProductName, UnitPrice
FROM Products
WHERE UnitPrice > (SELECT avg(UnitPrice) FROM Products)
ORDER BY UnitPrice;
```
8
```
SELECT DISTINCT ProductName as Twenty_Most_Expensive_Products, UnitPrice
FROM Products AS a
WHERE 20 >= (SELECT COUNT(DISTINCT UnitPrice)
                FROM Products AS b
                WHERE b.UnitPrice >= a.UnitPrice)
ORDER BY UnitPrice desc;
```

9

```
SELECT Count(ProductName)
FROM Products
GROUP BY Discontinued;
```

10.

```
SELECT ProductName,  UnitsOnOrder , UnitsInStock
FROM Products
WHERE (((Discontinued)=False) AND ((UnitsInStock)<UnitsOnOrder));
```